

Skills: build, compose, test, share

Module 6 · package a procedure once, use it every time

Lab: L04 Skills: build, compose and test

OBJECTIVES

What you will be able to do

- Say when something should be a skill, and write a SKILL.md by hand
- Write a description that makes Claude pick the skill at the right moment
- Choose who may invoke it: you, Claude, or both
- Pass arguments and inject live data with !command
- Bundle scripts and reference files, and run a skill in a forked subagent
- Test a skill: does it trigger, is it correct, is it cheap
- Share a skill safely with your team

The problem: repeated instructions

Without a skill

Monday: paste the 6 add-a-field steps

Wednesday: paste them again, forget step 2

Friday: a teammate writes their own version

With a skill

```
/add-todo-field
```

- Written once, in .claude/skills/
- Same steps every time, for everyone
- Or Claude loads it when you just ask

A skill is a folder with a SKILL.md: a procedure or reference Claude loads only when it is needed.

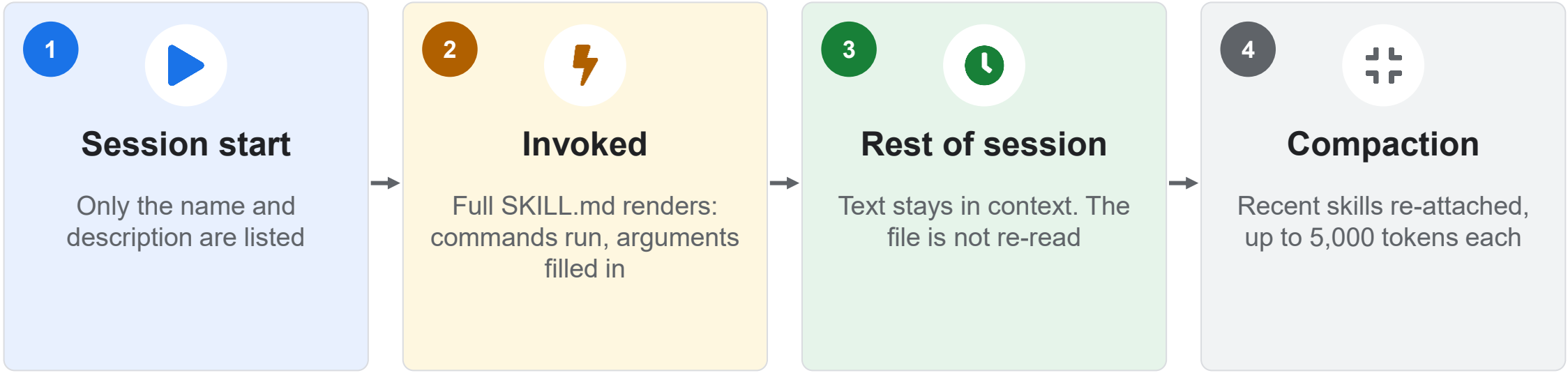
Skill or instruction file?

	INSTRUCTION FILE (M3)	SKILL
Loaded	Every session, in full	Description at start; full text only when used
Best for	Facts and rules needed always	Multi-step procedures and playbooks
Context cost	Constant	Small until invoked
Started by	Nobody: always present	You (/name) or Claude (matches the description)

Rule of thumb: a fact goes in CLAUDE.md; a procedure becomes a skill.

Old custom commands in .claude/commands/name.md still create /name. Skills are preferred: they add a folder, invocation control and automatic loading.

The life of a skill in a session



Progressive disclosure: description first, body when used, supporting files only when the body points to them.

So 30 skills cost little until one is used.

Where skills live, and which wins

SCOPE	LOCATION	SHARED WITH
Enterprise	Managed settings directory	Everyone on managed machines
Personal	`~/.claude/skills/<name>/SKILL.md`	You, in all projects
Project	`.claude/skills/<name>/SKILL.md`	The team, through git
Plugin	`<plugin>/skills/<name>/SKILL.md`	Whoever installs it, as /plugin:name (M13)

Same name?

Enterprise beats personal, personal beats project. Plugin skills have their own namespace, so they never clash. A project skill with a bundled skill's name replaces it.

Edits are live

Changes in an existing skills folder apply in the same session. Created the top-level skills folder after starting? Run /reload-skills.

Skills that come with Claude Code

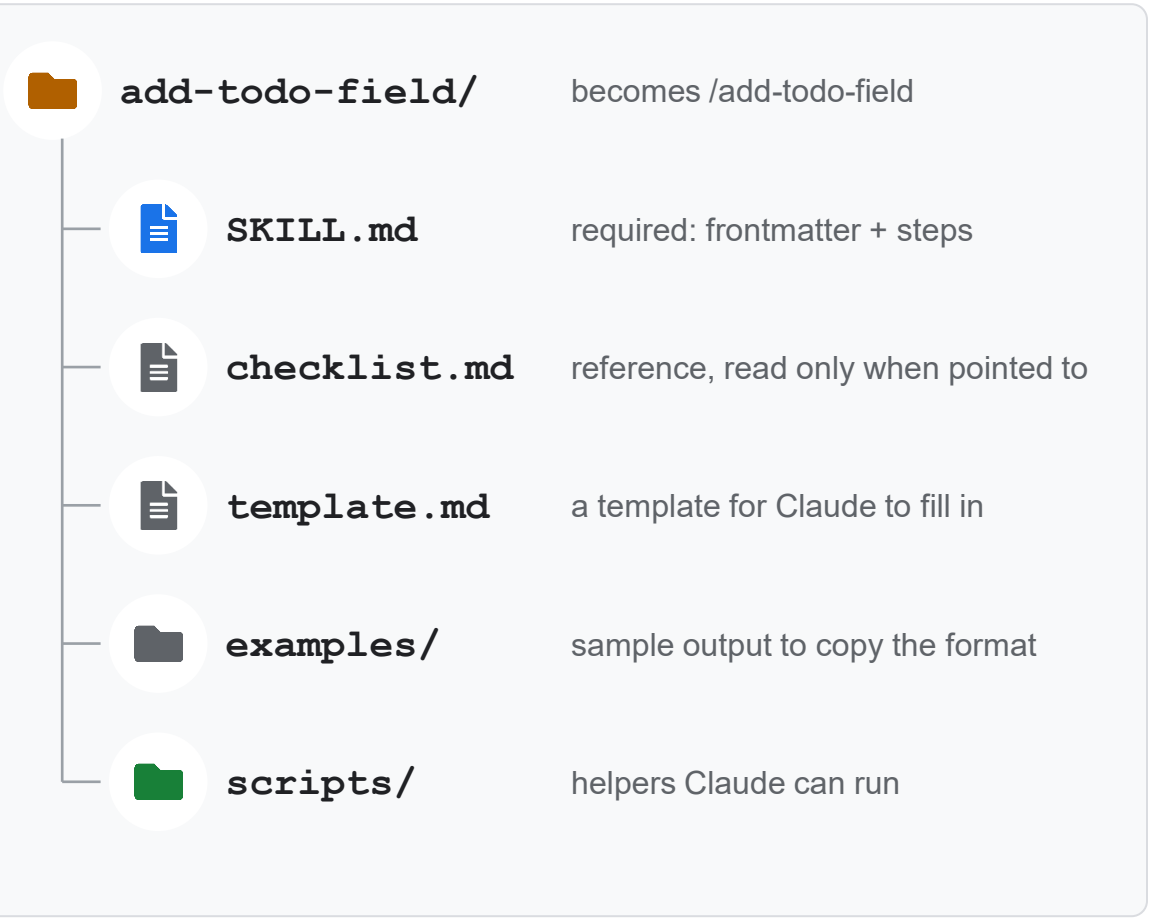
SKILL	USE IT FOR
`/simplify`	Review changed code for reuse and simplicity, then fix
`/code-review`	Review a diff or PR for bugs (M5)
`/batch`	Split a big change across worktrees (M5)
`/loop`	Repeat a prompt on an interval
`/debug`	Debug logging; diagnose the session

SKILL	USE IT FOR
`/verify`	Build and run the app to confirm a change works
`/run`	Launch and drive your app
`/doctor`	Check your setup and fix issues
`/skill-doctor`	Cost and usage of each skill
`/claude-api`	Claude API and SDK reference

Bundled skills are prompts, not fixed code: they give Claude instructions and let it use its tools. /init and /security-review are built-in commands.

Type /skills to see everything available, filter by typing, and see what each one costs.

Anatomy of a skill folder



Only SKILL.md is required

The folder name is the default /command name. Lowercase letters, numbers and hyphens.

Keep SKILL.md short

Under about 500 lines. Move long tables, policies and examples into other files and point to them. Claude reads them only when needed.

Inside SKILL.md

.CLAUDE/SKILLS/ADD-TODO-FIELD/SKILL.MD

```
---
name: add-todo-field
description: Adds a new field to to-dos, end to
  end. Use when asked to add a field, column
  or property to a to-do.
argument-hint: [field] [rules]
---
# Add a to-do field: $ARGUMENTS
1. Restate the field and its validation.
2. Write the failing pytest test first.
3. Change backend/app/models.py and main.py.
4. Update TodoForm.jsx and TodoItem.jsx.
5. Run uv run pytest -q.
6. Walk through checklist.md; report each item.
```



Frontmatter

Between the --- lines. Tells Claude what the skill is and when to use it.



Body

Plain markdown steps. Loaded only when the skill runs.

Frontmatter fields you will use

FINDING THE SKILL	PURPOSE	RUNNING IT	PURPOSE
`name`	The /command (defaults to folder)	`disable-model-invocation`	true: only you can run it
`description`	What it does and when to use it	`user-invocable`	false: hidden from / menu
`when_to_use`	Extra trigger words, added on	`allowed-tools`	Pre-approve tools while it runs
`argument-hint`	Autocomplete hint: [issue-number]	`disallowed-tools`	Remove tools while it runs
`arguments`	Named arguments: [name, folder]	`model, effort`	Override for that turn
`paths`	Globs that limit automatic use	`context, agent`	Run in a subagent (6.9)

All fields are optional; description is strongly recommended. Unknown keys are ignored.

6.4 · FRONTMATTER

Who can invoke a skill

SETTING	YOU	CLAUDE	DESCRIPTION IN CLAUDE'S CONTEXT?
(default: neither set)	**Yes**	**Yes**	Yes: Claude always knows it exists
`disable-model-invocation: true`	**Yes**	**No**	No: Claude never starts it on its own
`user-invocable: false`	**No**	**Yes**	Yes: background knowledge, not in the / menu

⚠ **If running it at the wrong moment would hurt (a deploy, a commit, a message), make it user-invoked only.**

How a description makes Claude pick a skill



PATTERN

```
<What it does>. Use when <situations, in the user's own words>.
```

Description + when_to_use is cut at 1,536 characters. When many skills compete, the least-used descriptions are dropped first. Put key words first.

Weak and strong descriptions

WEAK	STRONG
"Helps with to-dos"	"Adds a new field to to-dos, end to end. Use when asked to add a field, column or property to a to-do."
"Code stuff"	"Reviews a Python diff for missing pytest tests. Use before opening a pull request."
"React helper"	"Creates a React component in frontend/src with a test. Use when asked for a new component, page or widget."



Test it: ask for the task in plain words without naming the skill. If Claude does not load it, rewrite the description, not the body.

6.6 · YOUR TURN

Skill or not?

For each, choose: instruction file, skill (Claude may start it), skill (only you start it), or hook.

1. "Titles are 1 to 200 characters."
2. The 9-step procedure for adding a database migration.
3. "Publish release notes to the team channel."
4. "Format Python after every edit, no exceptions."

Passing arguments

WRITE	BECOMES
`$ARGUMENTS`	Everything typed after /name
`$ARGUMENTS[0], $0`	First argument (0-based)
`$name`	A named argument from arguments:
`${CLAUDE_SKILL_DIR}`	The folder holding SKILL.md
`${CLAUDE_PROJECT_DIR}`	The project root
`${CLAUDE_SESSION_ID}`	This session's id
`${CLAUDE_EFFORT}`	low ... max (current effort)

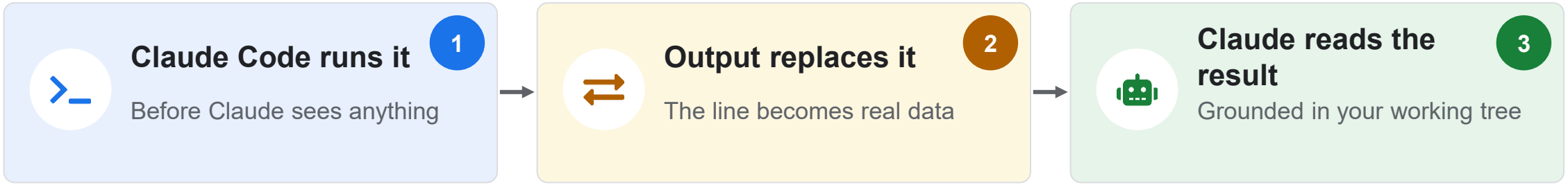
NEW-COMPONENT/SKILL.MD

```
---
name: new-component
description: Creates a React component
arguments: [name, folder]
argument-hint: [Name] [folder]
disable-model-invocation: true
---
Create frontend/src/$folder/$name.jsx
and a matching test. Follow the
style of TodoItem.jsx.
```

TYPE

```
/new-component PriorityBadge components
```

Live data: !command runs first



INLINE

```
## Current changes
!`git diff HEAD`
```

SEVERAL COMMANDS

````
```!
uv --version
python --version
git status --short
```
````

This is preprocessing, not a tool call. Each command is checked against your permission rules; a refused or failing command stops the skill. disableSkillShellExecution: true turns it off.

Example: my-changes

.CLAUDE/SKILLS/MY-CHANGES/SKILL.MD

```
---
name: my-changes
description: Summarises uncommitted changes and
  flags risky edits. Use when the user asks
  what changed or wants a commit message.
allowed-tools: Bash(git diff *)
---
## Current changes
!`git diff HEAD`

## Instructions
Summarise the changes in 2-3 bullets, then
list risks: local times instead of UTC,
titles in logs, missing tests. If the diff is empty, say so.
```

Live diff

The real git diff is pasted in before Claude starts.

No prompt

allowed-tools pre-approves git diff, so you are not asked.

allowed-tools pre-approves; it does not restrict

allowed-tools does

Pre-approve the listed tools while the skill runs, so you are not prompted for them.

vs

allowed-tools does not

Restrict anything. Every other tool stays callable, with your normal permission rules.

To take tools away, use disallowed-tools (6.10) or permission deny rules (M8).

Both are cleared when you send your next message, even though the skill text stays.

Scripts and reference files in a skill

PART	IN CONTEXT	USE FOR
SKILL.md	When the skill runs	Steps and must-not-miss rules
Reference .md	Only when Claude reads it	Long tables, policies, examples
Script	Its output only	Counting, parsing, validating
Asset	Only when read or copied	Templates, sample data

A deterministic step goes in a script: same answer every time, and only its output costs tokens.

IN SKILL.MD

```
Run python ${CLAUDE_SKILL_DIR}/
  scripts/list_todo_tests.py
  backend/tests
Then follow naming-rules.md.
```

IN THE FRONTMATTER

```
allowed-tools: Bash(python
  ${CLAUDE_SKILL_DIR}/scripts/*)
```

Review a skill's scripts like any program: they run with your permissions.

Skills that use other skills

What works

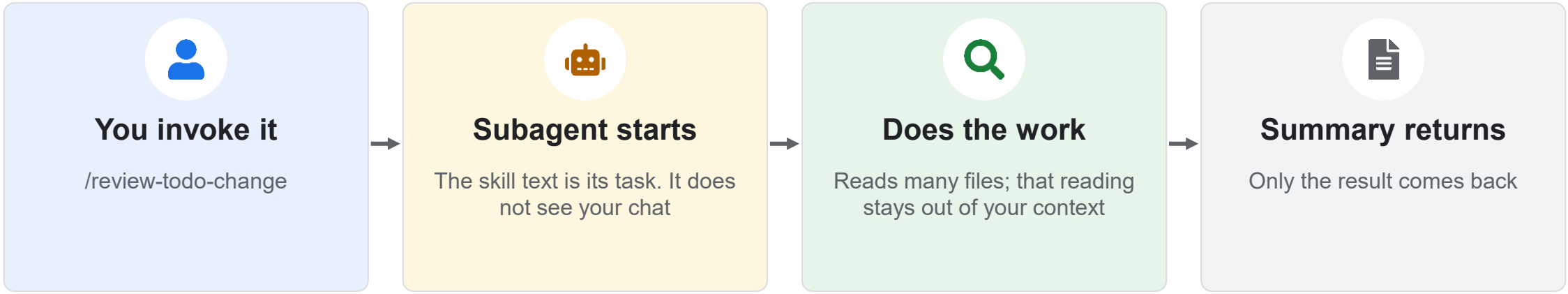
Say it in plain text: "For the tests, follow the add-todo-field skill." Claude loads that skill through its normal skill tool, if Claude is allowed to invoke it.

Two cautions

A skill with disable-model-invocation: true cannot be loaded by Claude, so no other skill can call it.
Every skill loaded stays in context: chaining three costs all three.

There is no special "call a skill" feature. When a step needs its own context or tools, use a forked skill (6.9) or a subagent (M7).

Run a skill in its own context: context: fork



FIELD	MEANING
`context: fork`	Run the skill in a subagent
`agent`	Which subagent: general-purpose (default), Explore (read-only), Plan, or your own from .claude/agents/ (M7)
`background`	true (default): keep working, the result arrives later. false: wait for it in this turn

Example: a forked, read-only review

.CLAUDE/SKILLS/REVIEW-TODO-CHANGE/SKILL.MD

```
---
name: review-todo-change
description: Reviews the branch's to-do
  changes in a separate, read-only context.
disable-model-invocation: true
context: fork
agent: Explore
background: false
allowed-tools: Bash(git diff *)
---
Diff to review:
!`git diff main...HEAD`

Check rules R1-R8 from CLAUDE.md, each
as OK or problem: UTC dates, titles 1-200,
no delete, no clock in rules.py, no
titles in logs, overdue boundary tested.
```



Explore

Read-only tools. It cannot edit your files.



background: false

You wait for the report in this turn.

Fork or stay inline?

No conversation

The subagent sees only the skill text. Write a full task, not "review what we discussed".

No project memory

Explore and Plan start without your CLAUDE.md. State key rules in the skill.

!command runs first

Before the subagent starts. Pre-approve it with allowed-tools.

Fork when

- The work is read-heavy (logs, many files)
- It must be read-only
- You only need the result

Stay inline when

- The work needs the conversation
- It edits files with your approval
- It is short guidance, like "dates are UTC"

6.9 · FORKED SKILLS

Skills and subagents

	SKILL	SUBAGENT (M7)
What it is	Packaged instructions and knowledge	A separate worker with its own context
Context	Joins your chat (unless context: fork)	Its own; sees none of your chat
Started by	/name, or Claude matching the description	Claude delegates, or you ask by name
Output	Stays in the conversation	Only its final message returns
Tools	Yours; allowed-tools / disallowed-tools adjust	Its own tool list
Live data	!command before Claude sees it	Runs Bash itself
Extras	Supporting files and scripts	memory, isolation: worktree, background
Portable	Agent Skills open standard	Claude Code only



Together: a subagent's skills: field preloads the full text of chosen skills when it starts (M7).

Restrict tools, add a hook

FRONTMATTER

```
---
name: todo-test-audit
description: Audits test names ...
disallowed-tools: Edit Write
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "python .claude/hooks/check.py"
---
```

FIELD	EFFECT
`disallowed-tools`	Removes tools while it runs. Cleared on your next message
`hooks`	Registered when invoked; stay for the rest of the session
`once: true`	On a hook: removed after its first run

A skill is a request; a hook is enforcement. disallowed-tools is not a security boundary (Claude could write through a shell). Hard rules go in permissions (M8) or hooks (M9).

What stays in context

SITUATION	WHAT HAPPENS
Same skill again, same rendered text	A short "already loaded" note, not a second copy
Same skill, new arguments or command output	The full text is added again
Auto-compaction or /compact (M3)	Recent skills re-attached after the summary (next slide)
allowed-tools / disallowed-tools	Cleared on your next message

1 Rules at the top

What must not be lost goes first in SKILL.md.

2 Standing instructions

"Run the tests after every edit", not "Run the tests".

3 Re-invoke or hook

Ignored after a long session? Invoke it again. Must always hold? Hook.

The compaction budget

25,000 TOKENS SHARED BY ALL RE-ATTACHED SKILLS · UP TO 5,000 EACH



One skill

Only its first 5,000 tokens come back

All skills

Share 25,000 tokens

Order

Filled from the most recently invoked

Anything past about 5,000 tokens of a skill may be cut after compaction. Illustrative: five skills of 5,000 tokens fill the budget.

Testing a skill: three questions

Seeing a skill load does not prove it is right.

Does it trigger?

3 prompts that should load it, in the user's words, and 3 that should not. Fresh session each. Wrong? Fix the description.

Is it correct?

Run a real task with the skill, then again with it off (skillOverrides: "off"). Compare the results and the diffs.

Is it cheap?

/context shows the skill listing size. /skill-doctor shows each skill's cost and use, and flags skills never used.

The skill-creator skill can automate trigger tests and A/B comparisons. Optional.

Tools that help, and "it never triggers"

COMMAND	USE
`/skills`	List, toggle, token cost
`/context`	The Skills row of your context
`/skill-doctor`	Cost and usage report
`/hooks`	Are the skill's hooks registered?
`claude --debug`	Frontmatter errors, budget warnings
`claude plugin validate .claude/skills`	Finds frontmatter that does not parse

/name works, but Claude never picks it

- Malformed YAML: the command still works, the description is not matched
- Description too vague: rewrite it in the user's words
- disable-model-invocation: true is set
- Listing over budget (about 1% of the context window): least-used descriptions dropped

Sharing a skill

Your team, one repository

Commit .claude/skills/ to git. Everyone who clones gets it. Review it like code.

You, all projects

Copy the folder to ~/.claude/skills/.

Many repositories or the company

A plugin (M13) or managed settings.

An open standard

Skills follow Agent Skills (agentskills.io), used by other AI tools too. Outside Claude Code only name, description, license, compatibility, metadata and allowed-tools are portable.

Community skills: read first

Anthropic's examples (skill-creator, frontend-design, webapp-testing) and community sets are useful. A skill is instructions plus scripts that run with your permissions. What does it run, touch, send?

Before you share

A skill folder is portable: copy it and it works, if every path uses ${CLAUDE_SKILL_DIR} and not a path on your machine.

Scripts

Touch only what they say they touch

allowed-tools

As narrow as the task

Description

Does not collide with another skill

Invocation

Side effects? disable-model-invocation: true

Never commit secrets or real user data inside a skill folder.

6.14 · YOUR TURN

Fork or inline?

For each, choose fork or inline.

1. "Read the whole backend/ folder and list every place a date is created or converted."
2. "Fix the failing test, using the steps in the skill."
3. "Review this branch's diff against the checklist; do not change anything."
4. A skill that only says "Always store dates in UTC."

REFERENCE

Skills that only you start

COMMIT/SKILL.MD

```
---
name: commit
description: Stage changes and create a
  conventional commit.
disable-model-invocation: true
allowed-tools: Bash(git add *)
  Bash(git commit *) Bash(git status *)
  Bash(git diff *)
---
1. git status, then git diff HEAD.
2. Stage the changes.
3. Message: <type>(<scope>): <summary>
   types feat, fix, docs, refactor,
   test, chore; under 72 characters.
4. Commit; show the hash.
Extra context: $ARGUMENTS
```

DEPLOY/SKILL.MD

```
---
name: deploy
description: Deploy the to-do app.
disable-model-invocation: true
argument-hint: [staging|production]
---
Deploy to $ARGUMENTS:
1. Must be staging or production.
2. cd backend && uv run pytest -q.
   STOP if any test fails.
3. cd frontend && npm run build.
4. Migrations: dry run first for
   production.
5. Deploy, then run smoke tests.
6. Report success or rollback steps.
Session: ${CLAUDE_SESSION_ID}
```

REFERENCE

A script skill and a research skill

LOG-SUMMARY/SKILL.MD (+ SCRIPTS/COUNT_ERRORS.PY)

```
---
name: log-summary
description: Summarises an application
  log. Use when the user asks what went
  wrong in a log or wants error counts.
allowed-tools: Bash(python
  ${CLAUDE_SKILL_DIR}/scripts/*)
---
Never paste log lines with tokens or
to-do titles; report counts and ids.
1. Run python ${CLAUDE_SKILL_DIR}/
   scripts/count_errors.py $ARGUMENTS
2. Classify the top 3 errors using
   severity-levels.md.
3. Say which one to look at first.
```

MAP-USAGE/SKILL.MD

```
---
name: map-usage
description: Finds every place a
  function or setting is used.
context: fork
agent: Explore
background: false
argument-hint: [name]
---
Find every use of $ARGUMENTS in this
repository with Glob and Grep.
Return file:line with one line of
context each, grouped by folder
(backend/, frontend/src/).
Do not edit anything.
```

REFERENCE

Switching skills off, and commands

.CLAUDE/SETTINGS.LOCAL.JSON

```
{
  "skillOverrides": {
    "log-summary": "off"
  }
}
```

VALUE	EFFECT
`"on"`	Normal (the default)
`"name-only"`	Listed without description
`"user-invocable-only"`	Hidden from Claude; / menu only
`"off"`	Hidden from both

COMMAND	DOES
`/skills`	List, filter, toggle, cost
`/reload-skills`	Pick up a new skills folder
`/context`	Size of the skill listing
`/skill-doctor`	Cost and usage report
`/hooks`	Skill hooks registered?
`/compact`	Summarise; check what a skill kept
`claude --debug`	Parse errors, budget warnings

CHECK YOUR UNDERSTANDING

Question 1

Which part of a skill does Claude read to decide whether to use it, and when does it read the rest?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

A skill that deploys should run only when you say so. Which field, and what else does that field change?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

A skill runs a 40-line Python script. What enters Claude's context: the source, or something else?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

Your forked review skill says "Review the changes we just discussed." What goes wrong?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 5

After a long session and a compaction, Claude ignores a rule at the bottom of an 8,000-token SKILL.md. Why, and what do you change?

Think first, then we discuss.